

An implicit feedback recommender for the Movielens dataset

Implicit feedback

For some time, the recommender system literature focused on explicit feedback: the Netflix prize focused on accurately reproducing the ratings users have given to movies they watched.

Focusing on ratings in this way ignored the importance of taking into account which movies the users chose to watch in the first place, and treating the absence of ratings as absence of information.

But the things that we don't have ratings for aren't unknowns: we know the user didn't pick them. This reflects a user's conscious choice, and is a good source of information on what she thinks she might like.

This sort of phenomenon is described as data which is missing-not-at-random in the literature: the ratings that are missing are more likely to be negative precisely because the user chooses which items to rate. When choosing a restaurant, you only go to places which you think you'll enjoy, and never go to places that you think you'll hate. What this leads to is that you're only going to be submitting ratings for things which, a priori, you expected to like; the things that you expect you will not like you will never rate.

This observation has led to the development of models that are suitable for implicit feedback. LightFM implements two that have proven particularly successful:

- BPR: Bayesian Personalised Ranking [1] pairwise loss. Maximises the prediction difference between a positive example and a randomly chosen negative example. Useful when only positive interactions are present and optimising ROC AUC is desired.
- WARP: Weighted Approximate-Rank Pairwise [2] loss. Maximises the rank of positive examples by repeatedly sampling negative examples until rank violating one is found. Useful when only positive interactions are present and optimising the top of the recommendation list (precision@k) is desired.

This example shows how to estimate these models on the Movielens dataset.

[1] Rendle, Steffen, et al. “BPR: Bayesian personalized ranking from implicit feedback.” Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence. AUAI Press, 2009.

[2] Weston, Jason, Samy Bengio, and Nicolas Usunier. “Wsabie: Scaling up to large vocabulary image annotation.” IJCAI. Vol. 11. 2011.

Getting the data

The first step is to get the [Movielens data](#). This is a classic small recommender dataset, consisting of around 950 users, 1700 movies, and 100,000 ratings. The ratings are on a scale from 1 to 5, but we’ll all treat them as implicit positive feedback in this example.

Fortunately, this is one of the functions provided by LightFM itself.

```
import numpy as np

from lightfm.datasets import fetch_movielens

movielens = fetch_movielens()
```

This gives us a dictionary with the following fields:

```
for key, value in movielens.items():
    print(key, type(value), value.shape)
```

```
('test', <class 'scipy.sparse.coo.coo_matrix'>, (943, 1682))
('item_features', <class 'scipy.sparse.csr.csr_matrix'>, (1682, 1682))
('train', <class 'scipy.sparse.coo.coo_matrix'>, (943, 1682))
('item_labels', <type 'numpy.ndarray'>, (1682,))
('item_feature_labels', <type 'numpy.ndarray'>, (1682,))
```

```
train = movielens['train']
test = movielens['test']
```

The `train` and `test` elements are the most important: they contain the raw rating data, split into a train and a test set. Each row represents a user, and each column an item. Entries are ratings from 1 to 5.

Fitting models

Now let's train a BPR model and look at its accuracy.

We'll use two metrics of accuracy: precision@k and ROC AUC. Both are ranking metrics: to compute them, we'll be constructing recommendation lists for all of our users, and checking the ranking of known positive movies. For precision at k we'll be looking at whether they are within the first k results on the list; for AUC, we'll be calculating the probability that any known positive is higher on the list than a random negative example.

```
from lightfm import LightFM
from lightfm.evaluation import precision_at_k
from lightfm.evaluation import auc_score

model = LightFM(learning_rate=0.05, loss='bpr')
model.fit(train, epochs=10)

train_precision = precision_at_k(model, train, k=10).mean()
test_precision = precision_at_k(model, test, k=10).mean()

train_auc = auc_score(model, train).mean()
test_auc = auc_score(model, test).mean()

print('Precision: train %.2f, test %.2f.' % (train_precision, test_precision))
print('AUC: train %.2f, test %.2f.' % (train_auc, test_auc))
```

```
Precision: train 0.59, test 0.10.
AUC: train 0.90, test 0.86.
```

The WARP model, on the other hand, optimises for [precision@k](#)—we should expect its performance to be better on precision.

```
model = LightFM(learning_rate=0.05, loss='warp')

model.fit_partial(train, epochs=10)

train_precision = precision_at_k(model, train, k=10).mean()
test_precision = precision_at_k(model, test, k=10).mean()

train_auc = auc_score(model, train).mean()
test_auc = auc_score(model, test).mean()

print('Precision: train %.2f, test %.2f.' % (train_precision, test_precision))
print('AUC: train %.2f, test %.2f.' % (train_auc, test_auc))
```

```
Precision: train 0.61, test 0.11.
AUC: train 0.93, test 0.90.
```

And that is exactly what we see: we get slightly higher [precision@10](#) (but the AUC metric is also improved).